

Spring Boot + Microservices

Vivek Gohil

What are Microservices ?

- To understand microservices, lets imagine a bank called EazyBank.
- Typically, banks comprise various departments, including Accounts, Cards, Loans.



Cards



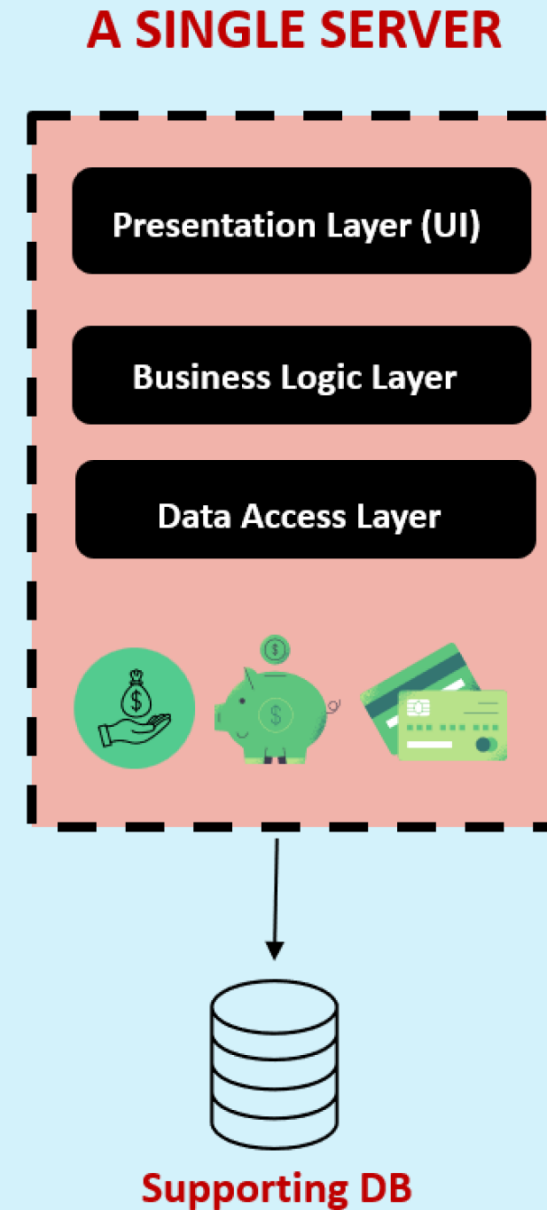
Loans



Accounts

The Monolith

- Back a decade, all the applications used to be deployed as a single unit where all functionality deployed together inside a single server.
- We call this architecture approach as Monolithic.



The Monolith

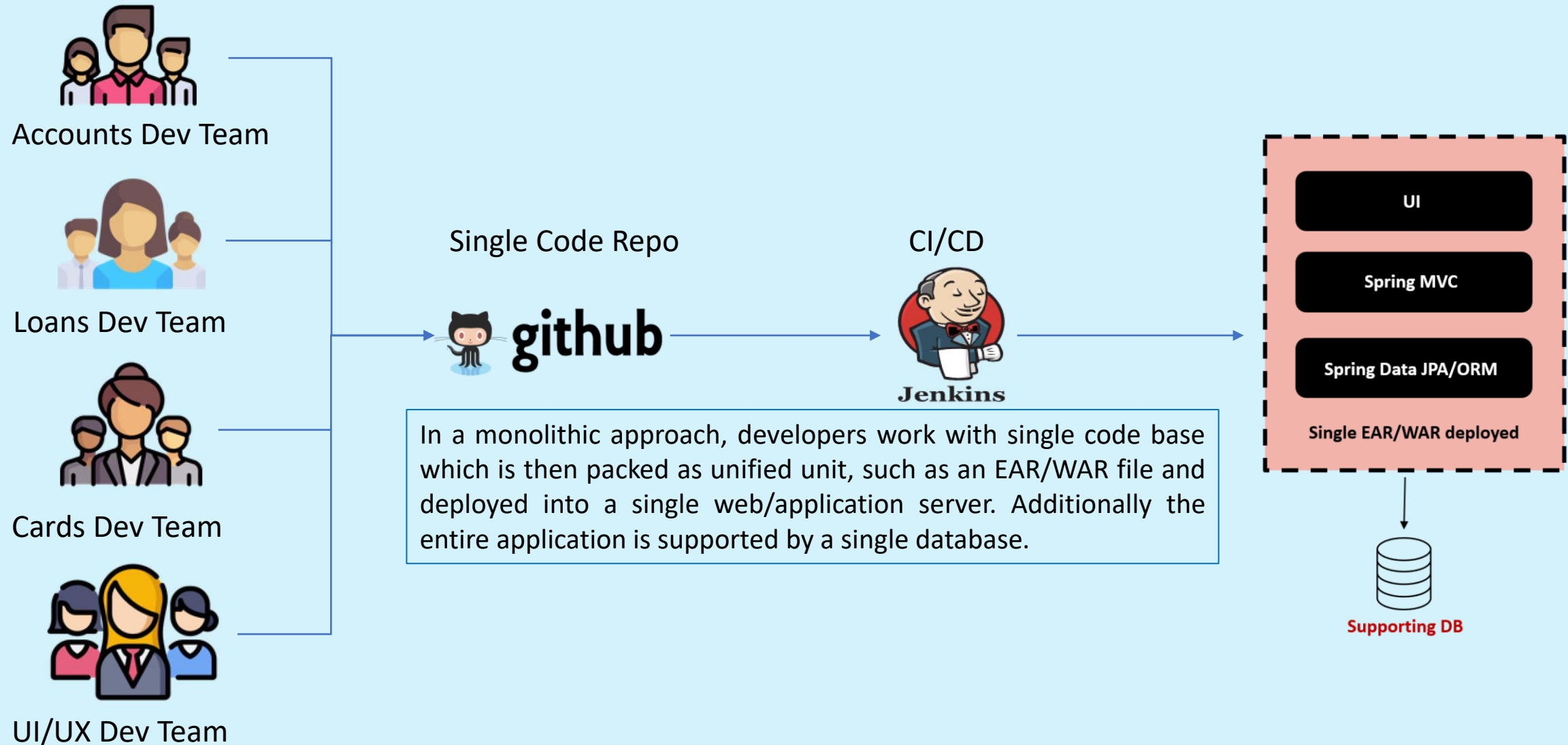
Pros

- Simple development and deployment for smaller teams and applications
- Fewer cross-cutting concerns
- Better performance due to no network latency

Cons

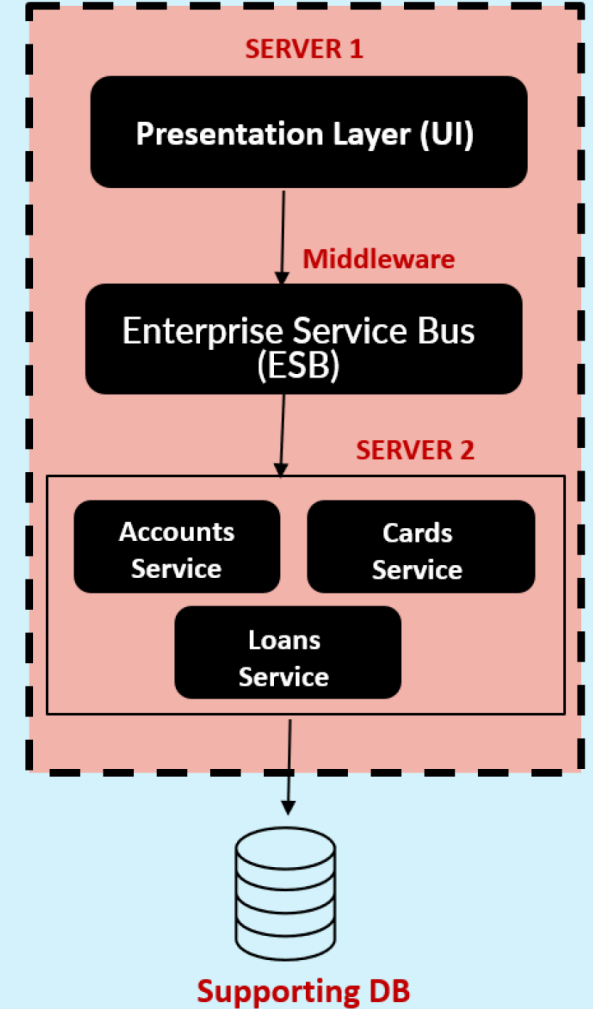
- Difficult to adopt new technologies
- Limited agility
- Single code base and difficult to maintain
- No fault tolerance
- Tiny update and feature development always need full deployment

The Monolith



The SOA (Service-Oriented Architecture)

- SOA emerged as an approach to combat the challenges of large, monolithic applications.
- It is an architectural style that focuses on organizing software systems as a collection of loosely coupled, interoperable services.
- It provides a way to design and develop large-scale applications by decomposing them into smaller, modular services that can be independently developed, deployed and managed.



The SOA (Service Oriented Architecture)

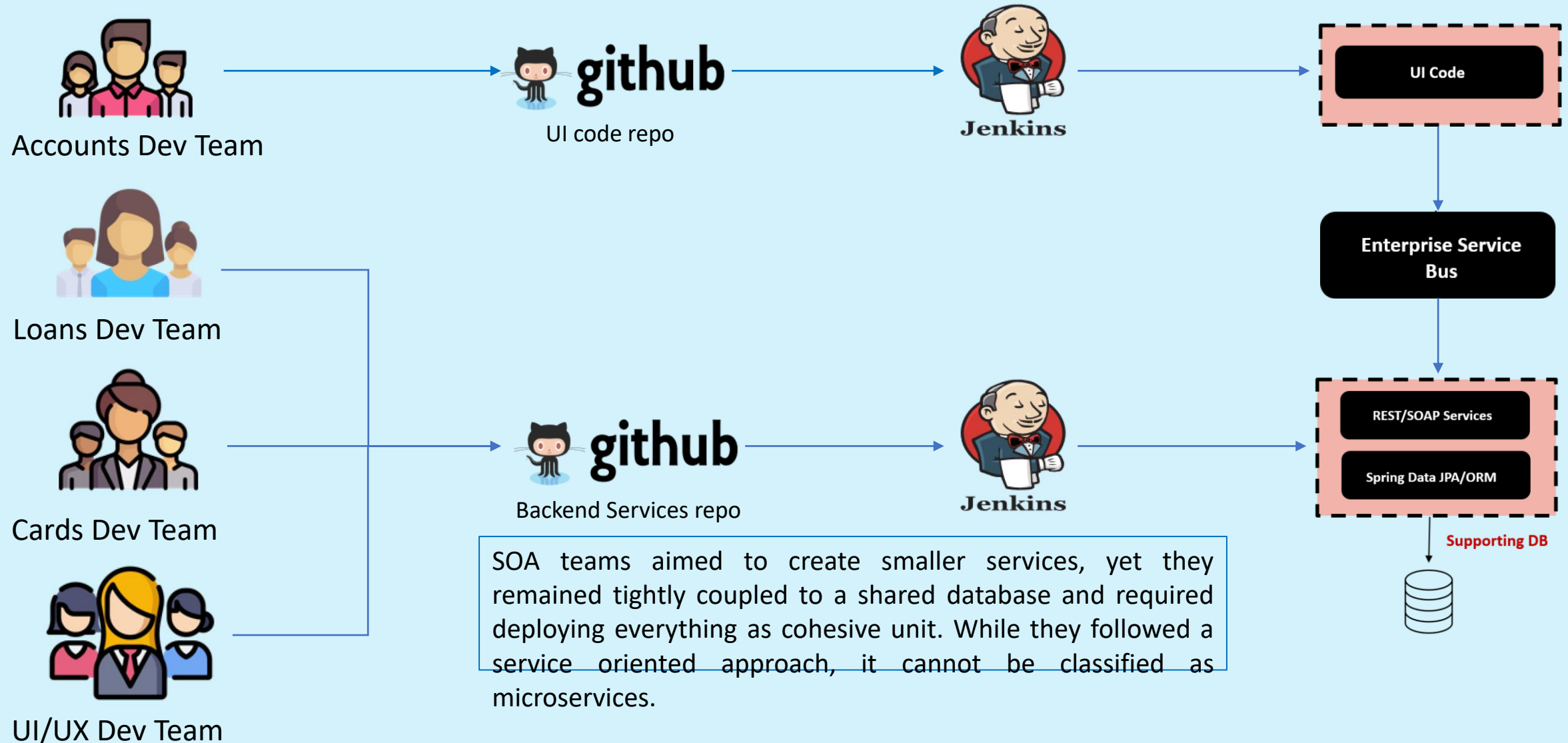
Pros

- Reusability of services
- Better maintainability
- Higher reliability
- Parallel development

Cons

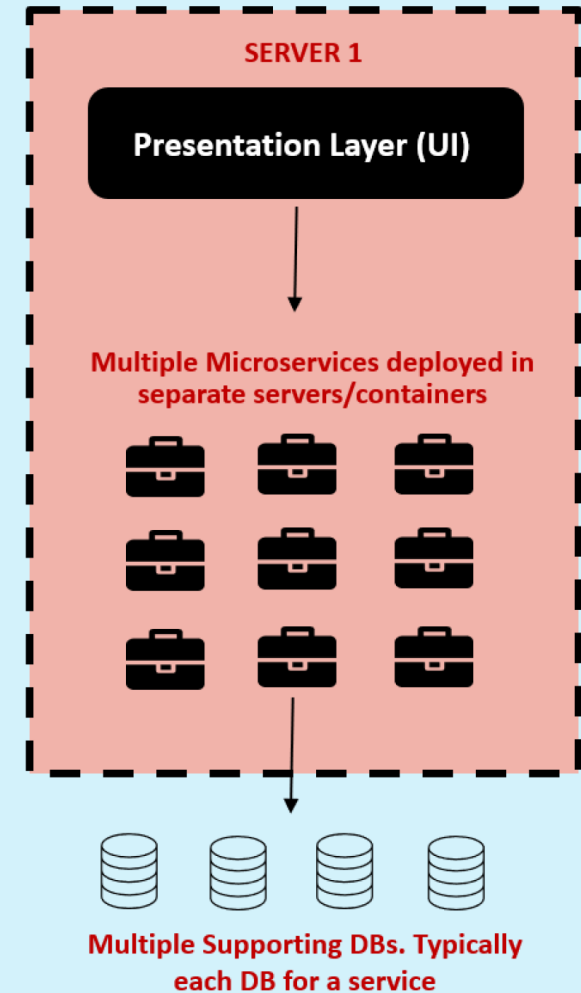
- Complex management due to communication protocols(SOAP).
- High investment costs due to vendor in middleware
- Extra overload

The SOA (Service Oriented Architecture)



The Great Microservices

- Microservices are independently releasable services that are modelled around business domain.
- A service encapsulates functionality and makes it accessible to other services via networks – you construct more complex systems from these building blocks.
- One microservice might represents Accounts, another Cards and yet another Loans, but together they might constitute an entire bank system.



The Great Microservices

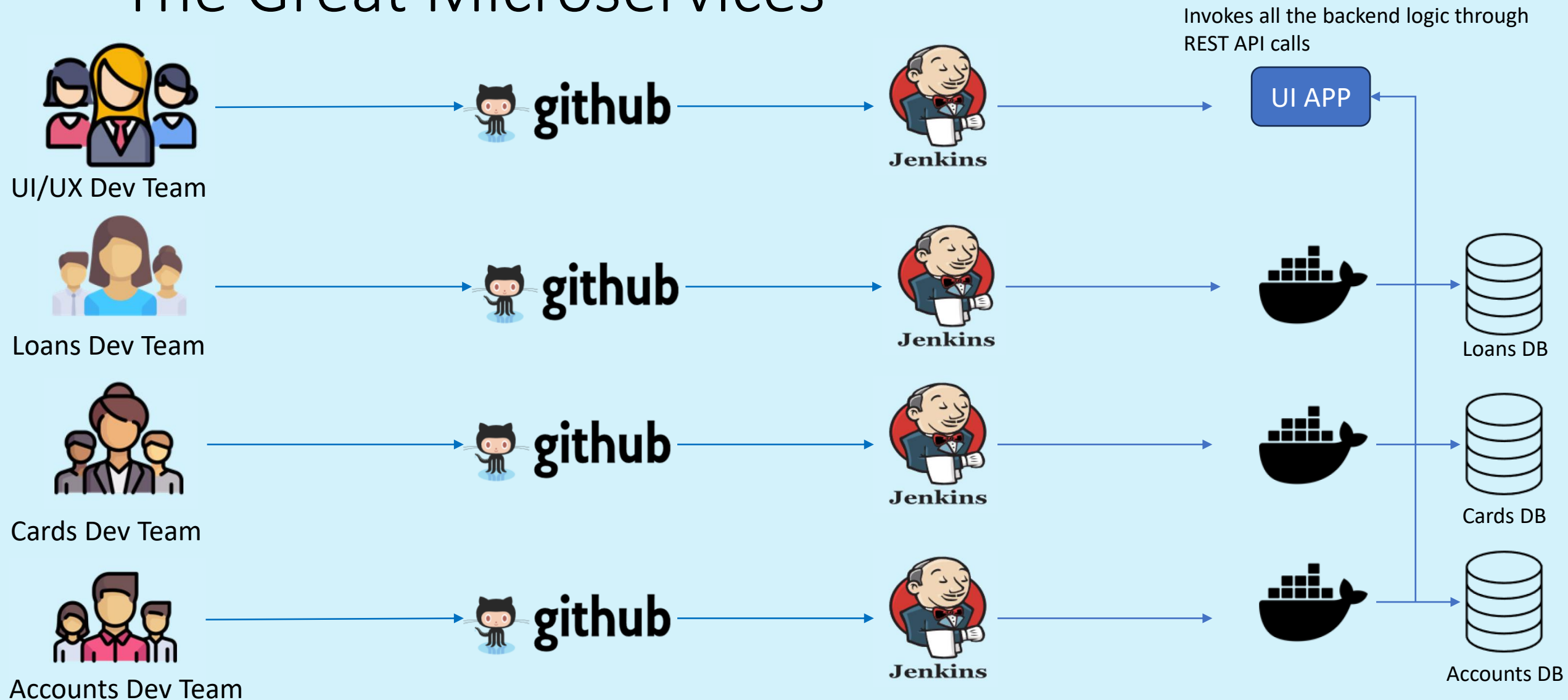
Pros

- Easy to develop, test and deploy
- Increased agility
- Ability to scale horizontally
- Parallel development
- Modelled around a business domain

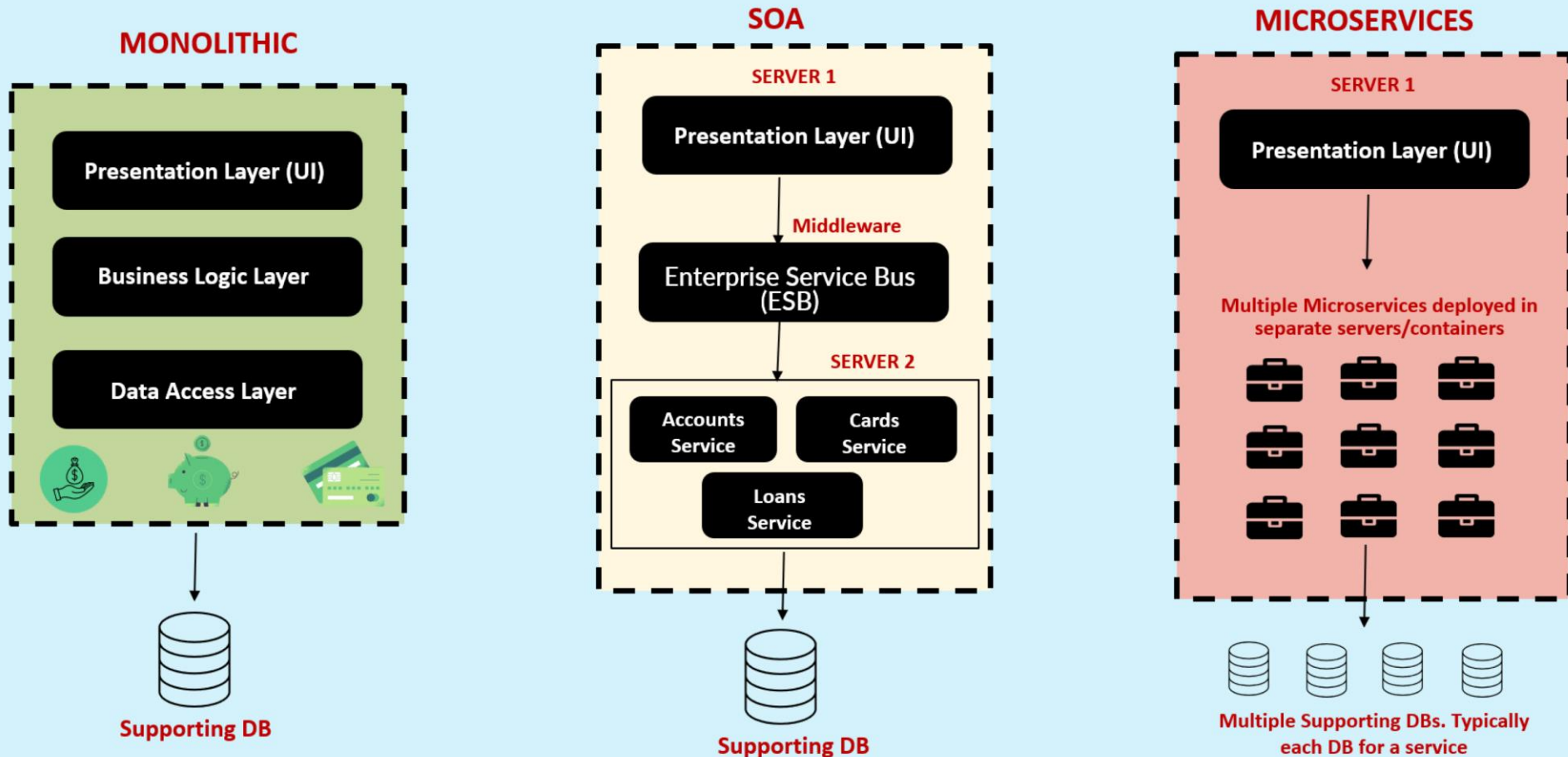
Cons

- Complexity
- Infrastructure overhead
- Security concerns

The Great Microservices



Monolithic vs SOA vs Microservices

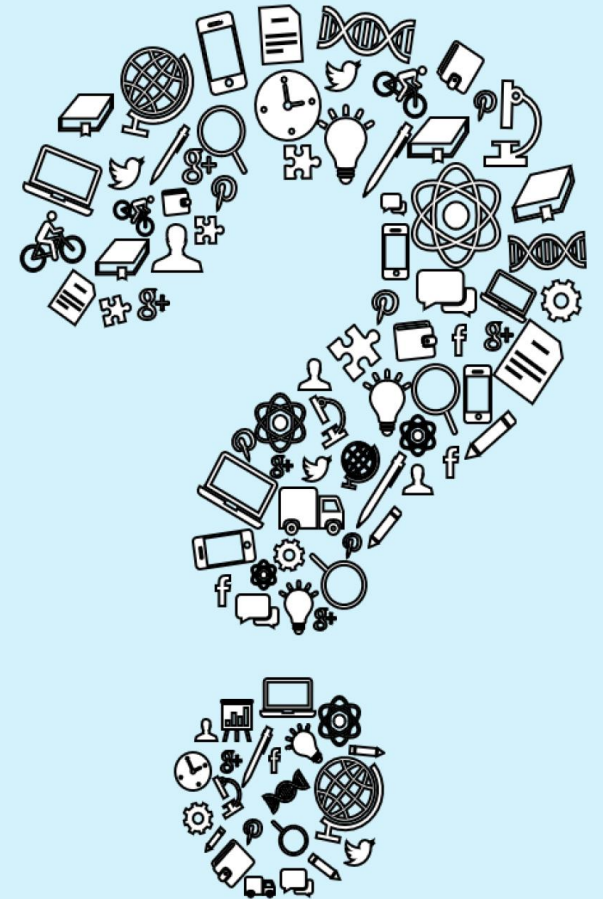


Monolithic vs SOA vs Microservices

FEATURES	MONOLITHIC	SOA	MICROSERVICES
Parallel Development			
Agility			
Scalability			
Usability			
Complexity & Operational overhead			
Security Concerns & Performance			

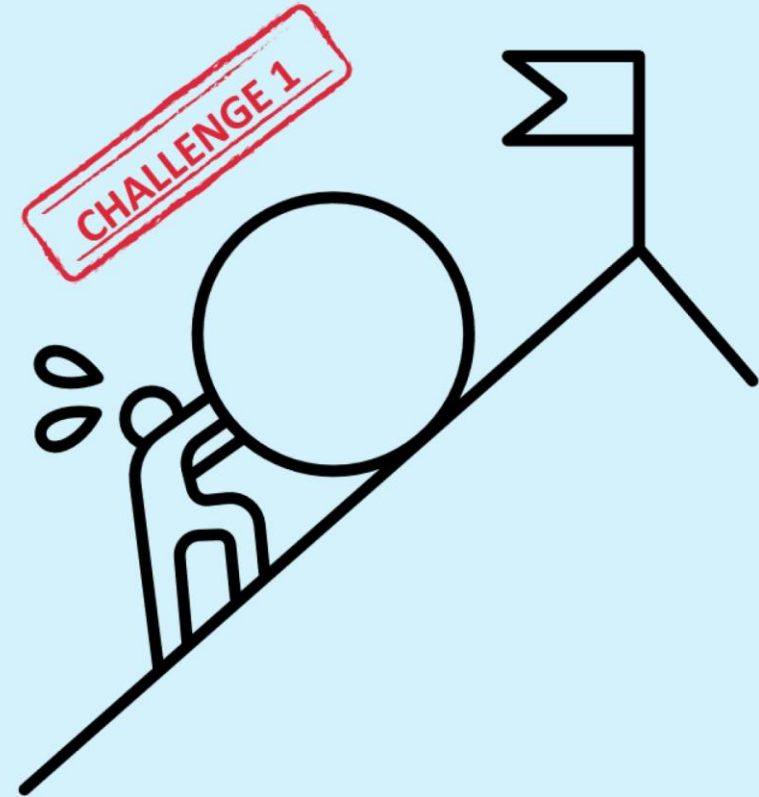
Definition of Microservices ?

- “Microservices is an approach to developing a single application as a suite of small services, each running in its own process and communicating with lightweight mechanisms, built around business capabilities and independently deployable by fully automated deployment machinery.” – From Article by James Lewis and Martin Fowler’s



How to build Microservices ?

- When considering a web application, the traditional approach involves packaging it as WAR or EAR file.
- These archive formats are commonly used to build Java applications, which are then deployed to web servers like Tomcat.
- Do you think the same approach works for building microservices ?



How to build Microservices ?

- Off course not, because organizations may need to build 100s of microservices. Building, packaging and deploying all the microservices using traditional methods can be an extremely challenging and practically impossible task.
- How to overcome this challenge ?
- The clue is Spring Boot



Why Spring Boot for Microservices ?

- Spring Boot is a framework that simplifies the development and deployment of Java/Spring applications, including microservices.
- With Spring Boot, you can build self-contained, executable JAR files instead of traditional WAR or EAR files.
- These JAR files contain all the dependencies and configurations required to the microservices.
- This approach eliminates the need for external web servers.

Why Spring Boot for Microservices ?

Provides a range of built-in features and integrations such as auto-configuration, dependency injection, and support for various cloud platforms



Provides an embedded Tomcat, Jetty, or Undertow server, which can run the microservice directly without the need for a separate server installation



Inbuilt support of production-ready features such as metrics, health checks, and externalized configuration



We can quickly bootstrap a microservice project and start coding with range of starter dependencies that provide pre-configured settings for various components such as databases, queues etc

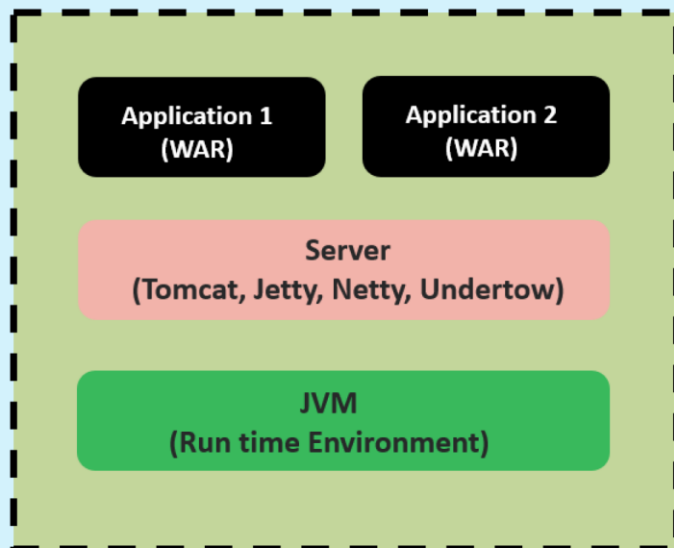


Well-suited for cloud-native development. It integrates smoothly with cloud platforms like Kubernetes, provides support for containerization, and enables seamless deployment to popular cloud providers



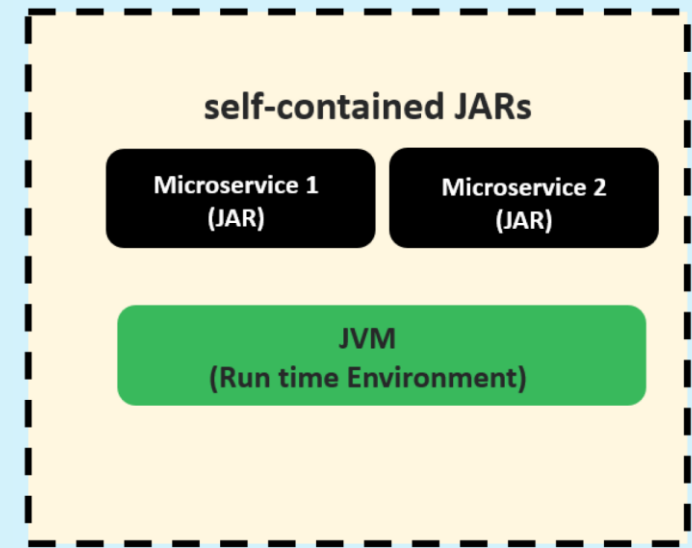
Why Spring Boot for Microservices ?

- In the traditional approach, applications are typically packed as WARs and rely on the presence of a server in the execution environment for running. However, in the microservices paradigm, applications are packed as self-contained JARs, also called fat-JAR, since they contain the application itself, the dependencies and the embedded server



WARs with external server

(Traditional approach)



JARs with embedded server

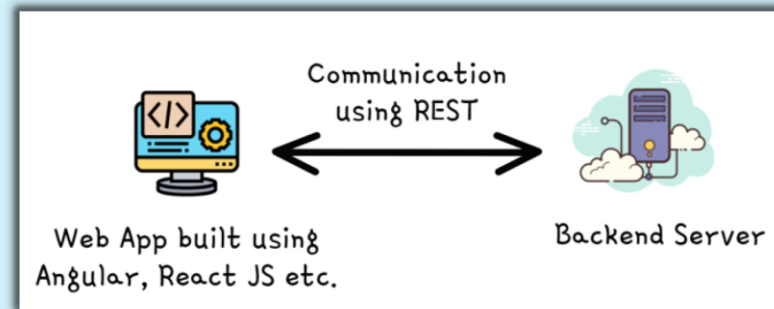
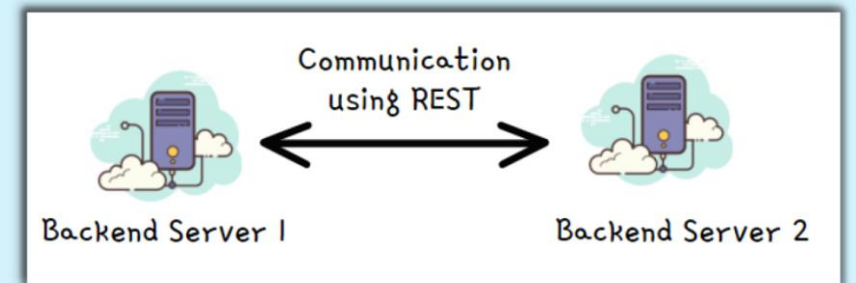
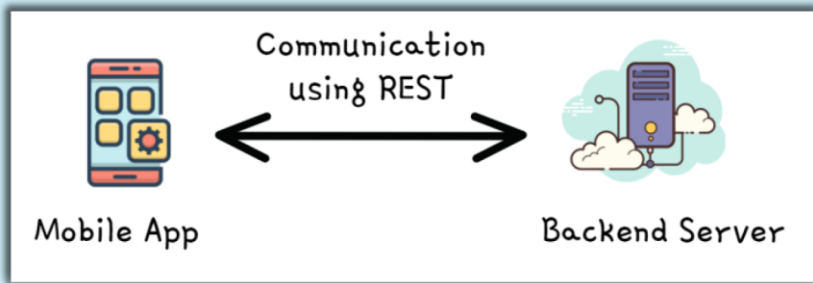
(Modern approach)

Implementing REST Services

- REST (Representational State Transfer) services are one of the most often encountered ways to implement communication between two web apps.
- REST offers access to functionality the server exposes through endpoints a client can call.

Implementing REST Service

- Below are the different use cases where REST services are being used most frequently these days.



Implementing REST Service

- Typically REST services are built to expose the business functionality and support CRUD operations on the storage system.
- Attached are the standards that we need to follow while build REST services.

Business logic supporting CRUD operations

Create -> HttpMethod.POST
Read -> HttpMethod.GET
Update -> HttpMethod.PUT/PATCH
Delete -> HttpMethod.DELETE

Proper input validation & exception Handling

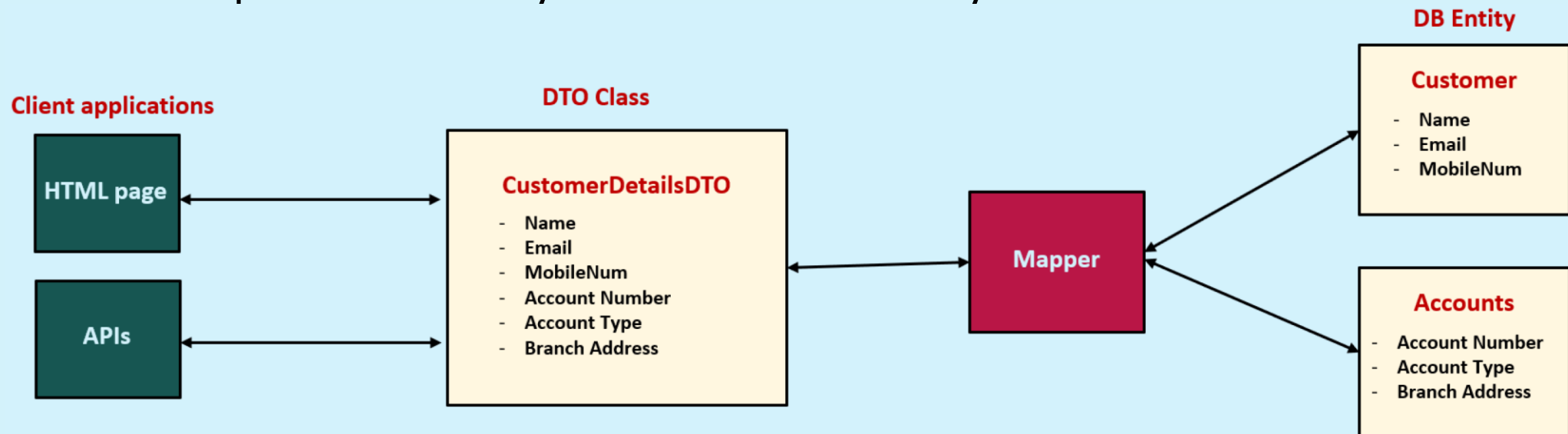
Make sure all the REST services perform input validations, handle the runtime and business exceptions properly. In all kind of scenarios, REST services should send a meaningful response to the clients

Document REST Services

With the help of standards like Open API Specification, Swagger, make sure to document your REST APIs. This helps Your client, third party developers to understand your services clearly.

DTO (Data Transfer Object) Pattern

- The Data Transfer Object(DTO) pattern is a design pattern that allows you to transfer data between different parts of your application.
- DTOs are simple objects that contain only data, and they do not contain any business logic.
- This makes them ideal for transferring data between layers of your application, such as the presentation layer and data access layer.



Different Annotations & Classes that supports building REST Services

@RestController – can be used to put on top of a call. This will expose your methods as REST APIs. Developers can also use **@Controller + @ResponseBody** for same behaviour

@ResponseBody – can be used on top of a method to build a Rest API when we are using **@Controller** on top of a Java class

ResponseEntity<T> - Allow developers to send response body, status, and headers on the HTTP response.

@ControllerAdvice – is used to mark the class as a REST controller advice. Along with **@ExceptionHandler**, this can be used to handle exceptions globally inside app. We have another annotation **@RestControllerAdvice** which is same as **@ControllerAdvice + @ResponseBody**

RequestEntity<T> - Allow developers to receive the request body, header in a HTTP request.

@RequestHeader & @RequestBody – is used to receive the request body and header individually.